



ЛЕКЦИЯ 3

Выполнила Ветрова Ольга
Авенировна, доцент каф. АСОИ и У

Основы работы с графическим языком *Visual LISP*

- Система **nanoCAD** – это графическая система для ПЭВМ, ориентированная на автоматизированное проектирование технических систем.
- Первым объектом освоения обычно является **графический редактор**, составляющий основу системы. Редактор позволяет создавать, исследовать и модифицировать геометрические модели объектов, в том числе пространственные (трехмерные), но главным образом предназначен для автоматизации выпуска чертежей и технической документации.
- **Любой графический редактор позволяет скомпоновать чертеж на экране графического дисплея с помощью устройств ввода ЭВМ и вывести его на бумажный носитель.** Однако специалист, освоивший графический редактор, обнаружит, что выигрыш во времени по сравнению с традиционным способом незначителен, а иногда его и вовсе нет.

Источники сокращения времени при работе с графическим редактором

- **Полуавтоматическая простановка** размеров, штриховки, условных обозначений, построения рамки и штампа.
- **Наличие в чертеже повторяющихся фрагментов** и в особенности в том случае, если эти фрагменты повторяются в различных чертежах. Такая ситуация характерна, например, для технической документации. Наибольший эффект достигается в том случае, когда чертеж, созданный с помощью графического редактора, в дальнейшем неоднократно модифицируется, причем каждая модификация затрагивает небольшую часть чертежа, а выпускать новый чертеж приходится целиком.
- **Суммарное время по всем модификациям может оказаться в несколько раз меньше**, чем при ручной разработке.

Оценка возможностей системы nanoCAD по автоматизации графических работ и выпуска документов

- Графические изображения и чертежи можно получать **без затрат времени и усилий** на «рисование».
- Чертеж может быть **сформирован за минуты** (не считая времени автоматического вывода чертежа на принтер).
- Графическое изображение (документ) может получить специалист, **практически ничего не знающий** о компьютерной графике и графических системах.
- Но чтобы получить такой эффект, кто-то **предварительно должен написать программу для генерации семейства сходных чертежей**.
- Именно эту программу теперь может **применить любой пользователь** для получения любого изображения (чертежа), принадлежащего данному семейству.

Введение в язык графического программирования *Visual LISP*

- Для создания программ, выпускающих чертежи и документы, в системе naпoCAD имеется **мощное средство – язык графического программирования *Visual LISP***.
- Но чтобы использовать богатые возможности *Visual LISP*, нужно **научиться программировать на этом языке**.
- Язык графического программирования *Visual LISP* – это вариант языка логического программирования *LISP*, в который добавлены **функции доступа к объектам, таблицам и словарям графической системы**.
- Основой языка *LISP* является **работа со списками**, которые могут иметь произвольную длину и включать элементы разной природы (числа, текстовые строки, указатели файлов и т.д.).
- В то же время в языке доступны **обычные арифметические вычисления, логические операции, работа с файлами** и т.д.
- С помощью языка *Visual LISP* можно **писать программы** или **вводить в командной строке выражения**, которые затем вычисляет графическая система.

Понятие графической программы, формирующей изображение

- Рассмотрим **понятие графической программы, формирующей изображение**, отталкиваясь от графического редактирования — уже знакомого нам способа формирования изображения.
- Этот способ заключается в следующем: **конструктор определяет характер очередного действия**; затем **вводит соответствующую команду в ЭВМ**; графическая **система интерпретирует эту команду**, т.е. разворачивает ее в последовательность команд процессора ЭВМ; затем эта **последовательность команд выполняется**; **результат выполнения отображается на экране**.
- Рассматриваемый способ характеризуется тем, что **конструктор мыслит дискретными командами**, а **вся их последовательность заранее не определена** и присутствует в его голове лишь как **общий замысел**, который **уточняется и корректируется в ходе создания изображения**.
- Программы на *Visual LISP* обладают **дополнительными возможностями** по сравнению с системой команд и позволяют **выполнять более сложные действия и записывать действия более компактно**.

Возможности языка *Visual LISP*, общие со всеми языками программирования высокого уровня

- обращение из программы к подпрограммам;
- компактная запись повторяющихся последовательностей действий — циклов;
- ввод данных пользователем в процессе выполнения программы;
- вывод сообщений из программы;
- проверка выполнения разного рода условий;
- выбор направления действий в зависимости от результатов проверки.

Особенности технологии программирования на языке *Visual LISP*

- Общая технология программирования включает этапы:
- постановку задачи, разработку алгоритма, написание программы, ввод программы в ЭВМ, отладку программы.
- Постановка задачи для технологии программирования на языке *Visual LISP*:
- подбор чертежей всех деталей данного семейства; составление на их основе чертежа комплексной детали;
- определение условий, которые влияют на выбор конкретной конфигурации детали из комплексной детали; определение постоянных размеров на чертеже;
- определение параметров, которые должны вводиться конструктором при работе с программой;
- описание соотношений, позволяющих получить все переменные параметры на основе параметров, вводимых конструктором;
- разделение текстовой части чертежа на постоянную и переменную части, определение правил ввода переменных частей текста и связывания постоянных и переменных частей;
- перечисление сообщений, которые программа должна выдавать конструктору.

Особенности технологии программирования на языке *Visual LISP* (продолжение)

- При **разработке алгоритма** решаются два следующих вопроса: **как будет устроена программа** и **как она будет выполняться**. На этом этапе определяется **состав модулей программы** и разрабатываются **алгоритмы каждого модуля**.
- Написание программы – **перевод алгоритма на язык программирования**.
- Программа тщательно **проверяется и отлаживается**.
- После того как программа отлажена, т.е. получены первые результаты, полностью удовлетворяющие пользователя, **работа программиста может считаться законченной**.
- Теперь программа, хранящаяся на магнитном диске, в любое время может быть **вызвана на выполнение любым пользователем**.
- В процессе выполнения программы **пользователь ведет диалог с ЭВМ, принимает сообщения программы и вводит в программу данные**.

Назначение языка *Visual LISP*

- Графический язык программирования *Visual LISP* является **расширением языка программирования *LISP***.
- *LISP* — это язык высокого уровня, ориентированный **на обработку списков**, который выбран в качестве базового потому, что **графические примитивы** (начиная, с точки), блоки, наборы примитивов и блоков удобно представляются **в виде списков**.
- В составе системы *naпoCAD* имеется *интерпретатор* с языка *Visual LISP*.

Классы прикладных задач для языка *Visual LISP*

Создание и использование графических баз данных

- Если накоплено большое количество чертежных файлов, программ на *Visual LISP*, соответствующих чертежным фрагментам, деталям, узлам, то их можно в некотором смысле считать графической базой данных.
- Программы на *Visual LISP* в сочетании с пользовательскими меню могут организовывать просмотр, поиск, подключение к объектам их частей и т.п. Тогда работа специалиста в графической системе будет сводиться к поиску нужных объектов или частей чертежа, обращению к соответствующим *LISP*-программам и ответам на вопросы этих программ.
- Хранение графических данных в виде набора программ на *Visual LISP* дает возможность в десятки и сотни раз сократить требуемую память на магнитном диске по сравнению с памятью, необходимой для хранения чертежных файлов, так как,
- во-первых, одна программа позволяет получить не один, а множество чертежей,
- во-вторых, текст программы на *Visual LISP* занимает на порядок меньше памяти, чем *DWG*-файл, который может быть получен в результате работы этой программы.

Программирование чертежей с параметризацией

- Применение параметров не имеет практически никаких ограничений при создании семейства графических изображений. Числовые значения параметров для графических объектов могут быть даже отрицательными.

Классы прикладных задач для языка Visual LISP (продолжение)

- **Анализ и (или) автоматическое преобразование** изображений.
- Программа также может быстро осуществить преобразование изображения, на которое при работе в графическом редакторе пришлось бы затратить значительное время. Например, заменить все **вставки одного типа на вставки другого типа** из какого-либо чертежного файла; **перенести все объекты с одного слоя на другой слой**; **повернуть все блоки на заданный угол** — каждый относительно своей базовой точки.
- Расширение системы команд графического редактора и построение на основе универсального редактора **специализированных автоматизированных систем**, имеющих гораздо более простой и естественный для пользователей язык, ориентированный на конкретную предметную область. В этом случае хорошим дополнением является **возможность создания пользовательских меню**.

Способы ввода программ на языке AutoLISP

- Программа на языке *Visual LISP* представляет собой построенный с помощью любого редактора текстов с соблюдением правил языка **текстовый файл**, который хранится на магнитном диске под определенным именем. Он может быть загружен в ОЗУ в ходе сеанса графического диалога в графической системе и может быть выполнен путем соответствующего ответа на подсказку системы:
- *COMMAND*: (в русской версии – *Команда*:)
- **Программа** – последовательность функций *Visual LISP* и функций, определенных самим программистом.
- Помимо функций в тексте программы могут присутствовать комментарии. **Комментарий** – это любой текст (он может быть и на русском языке), начинающийся со знака «;». Комментарием считается весь текст от знака «;» до конца строки. Комментарии не обрабатываются интерпретатором, а служат только для лучшего понимания текста программы.

Основные понятия языка *Visual LISP*

- Данные в *Visual LISP* принято называть *атомами*. Атомы – это элементарные объекты, из которых формируются выражения *Visual LISP*. Рассмотрим атомы следующих типов: константы целые, действительные, текстовые (строковые), а также переменные (идентификаторы, символические имена).
- Рассмотрим соглашения по форме записи правил и выражений языка *Visual LISP*. Будем описывать эти правила с помощью символов «< >» (угловые скобки), «::=» (двойное двоеточие со знаком равенства), «[]» (квадратные скобки), «...» (многоточие). Все эти символы не встречаются в выражениях *Visual LISP*, и поэтому их можно использовать для записи правил *Visual LISP*.
- **Символ «::=»** означает «это есть»;
- **Запись < объект >** означает, что на данное место можно подставить любой объект указанного класса.
- **Запись [< объект >]** означает, что объект может присутствовать, а может и не присутствовать в данном выражении.
- **Запись < объект >...** означает, что объектов данного класса может быть несколько.

ПРИМЕРЫ АТОМОВ

- $\langle \text{целое} \rangle ::= [-] \langle \text{цифра} \rangle \dots$
- Составной элемент программы на *Visual LISP*, состоящий из атомов или других списков, отделенных друг от друга пробелами, называется **списком**.
- $\langle \text{список} \rangle ::= ([\langle \text{элемент} \rangle \dots])$, где $\langle \text{элемент} \rangle ::= \langle \text{атом} \rangle$.
- Как следует из приведенной записи, возможен список и без элементов, т.е. список вида $()$. Такой список имеет значение **NIL** (пусто или пустое множество).
- Частным случаем списка является **функция**, определяемая следующим образом:
- $\langle \text{функция} \rangle ::= (\langle \text{имя функции} \rangle [\langle \text{аргумент} \rangle \dots])$
- Здесь $\langle \text{имя функции} \rangle$ – это **символическое имя**. Каждая функция имеет свое, связанное с ней имя.
- **Аргументы функции** – это выражения, построенные по правилам *Visual LISP*, в том числе другие функции. Для каждой конкретной функции характерны определенное количество, типы и порядок следования аргументов.

ПРИМЕРЫ ФУНКЦИЙ

- 1) (*COS* <угол>), где *COS* – имя функции для вычисления косинуса; <угол> – аргумент, который может быть действительной или целой константой или переменной.
- 2) (+ <число> <число>...), где знак “+” – имя функции для вычисления суммы; < число > – действительная или целая константа или переменная. Из записи следует, что аргументов этой функции может быть два и более.
- 3) (*COS*(+ABC)).
- В примере 3) функция сложения производит суммирование трех слагаемых, обозначенных именами *A*, *B*, *C*, и возвращает результат, имеющий форму числа и равный сумме $A+B+C$. Этот результат используется программой, а именно – подставляется в качестве аргумента в функцию *COS*. В свою очередь, функция *COS* возвращает свой результат (новое число), который тоже может быть использован. В дальнейшем при описании каждой конкретной функции будут приведены ее имя, характеристика аргументов и возвращаемое значение.

ОПИСАНИЕ НОВОЙ ФУНКЦИИ В ПРОГРАММЕ

- Описание новой функции в программе должно предшествовать обращению к этой функции. Описание новой функции в *Visual LISP* – это тоже функция, имени которой предшествует **ключевое слово *DEFUN***:
- <описание функции>::=(DEFUN <имя функции>) ([<аргумент>...]/[<локальная переменная>...]) <выражение>)*
- Программа 1. Вычисление суммы $SIN(A) + COS(A)$*
- (DEFUN SINCOS (A) ; имя функции и список аргументов*
- (+ (SIN A) (COS A)); выражение*
-) ;конец программы.*
- Программа 1 состоит из единственной функции *DEFUN*. Эта функция описывает придуманную нами функцию, которую мы назвали *SINCOS*.

ОПИСАНИЕ НОВОЙ ФУНКЦИИ В ПРОГРАММЕ (продолжение)

- Функция *SINCOS* имеет один аргумент, который мы обозначили именем *A*. Выражение *Visual LISP* в составе функции показывает, что должна делать данная функция.
- Для выполнения функции *SINCOS* можно в ответ на очередное приглашение графической системы набрать строку (*SINCOS 0*). Тем самым интерпретатору *Visual LISP* будет задана новая программа, состоящая из одной функции *SINCOS* с определенным значением аргумента, в нашем случае равным нулю. Эта функция возвращает число, равное $SIN(0)+COS(0)$, в нашем случае *1.0*. Это число будет отображено на экране дисплея.

07.10.2024

ВВОД И ВЫВОД РАЗЛИЧНЫХ ТИПОВ ДАННЫХ

Раздел 1

07.10.2024

***ПРИМЕР ПРОГРАММЫ ДЛЯ ЛАБОРАТОРНОЙ
РАБОТЫ № 5 «ЗНАКОМСТВО С ЯЗЫКОМ
ГРАФИЧЕСКОГО ПРОГРАММИРОВАНИЯ.
ПРОГРАММИРОВАНИЕ ВЫЧИСЛЕНИЙ»***

Раздел 1.1

Программа вычисления выражения

```
(defun lsp10_16 (B )  
;выражение  $50 - (B * 30) + 5 * (B - 80)$ .  
(+ (- 50 (* B 30)) (* 5 (- B 80))))  
)
```

07.10.2024

***РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ ПРОГРАММЫ ДЛЯ
ЛАБОРАТОРНОЙ РАБОТЫ № 5 «ЗНАКОМСТВО С
ЯЗЫКОМ ГРАФИЧЕСКОГО ПРОГРАММИРОВАНИЯ.
ПРОГРАММИРОВАНИЕ ВЫЧИСЛЕНИЙ»***

Раздел 1.2

Результат программы

```
Command: (lsp10_16 10.0)
```

```
-600.0
```

```
Command: (lsp10_16 5.0)
```

```
-475.0
```

```
Command: (lsp10_16 15.0)
```

```
-725.0
```

07.10.2024

ТИПЫ ДАННЫХ В VISUAL LISP

Раздел 1.3

LISP работает с данными следующих типов:

- ❖ целое число;
- ❖ вещественное число;
- ❖ строка;
- ❖ список, точечная пара, точечный список;
- ❖ дескриптор файла;
- ❖ указатель функции, приложения;
- ❖ примитив *NanoCAD*;
- ❖ набор.

Символам *LISP* могут быть присвоены значения любых типов.

- Символ языка *LISP* может в ходе программы менять тип своего значения.
- Пользователь в явном виде может задавать константы следующих типов:
- **целые, вещественные, строковые и списковые.**

Целая константа задается правилом:

- *< целая константа > ::= [+/-] < цифра > ...*
- Здесь +/- означает плюс или минус.
- Целые числа могут задаваться в пределах от -32768 до 32767.
- Целое число без знака — от 0 до 65535.

Действительная константа задается правилом:

- *<действительная константа >:: = [+/-]< цифра >< цифра >...*
- Здесь точка отделяет целую часть от дробной части.
- Для чисел с большими порядками удобна запись
- *[+/-]< цифра >....< цифра > E[-] < цифра > [< цифра >] .*
- Здесь *E* означает «10 в степени», а показатель степени ставится после *E*.

В *Visual LISP* определена также константа, обозначаемая *PI* и имеющая значение 3.1415926.

- Константу *PI* можно назвать предопределенным символом.
- К предопределенным символам в *LISP* относятся следующие:
 - ❖ *T* – единственная логическая константа, имеющая значение истина;
 - ❖ *NIL* – пустое значение, не эквивалентное числу 0 или пустой строке “”;
 - ❖ *PI* – постоянная, равная отношению длины окружности к диаметру;
 - ❖ *PAUSE* – постоянная, позволяющая ввести паузу при работе команды графического редактора (используется в функции *COMMAND*).
- Значения предопределенных символов пользователям не рекомендуется изменять.

Для текстовой константы можно использовать запись:

- *< текстовая константа > ::= "< символ >..."*.
- Длина текстовой константы не более 100 знаков.
- Если в тексте встречается знак «\», то он должен быть удвоен (т.е. вместо «\» должно быть записано «\\»).
- Это связано с тем, что символ «\» используется для ввода управляющих символов: \n — перевод строки, \r — возврат каретки, \t — табуляция.

Для переменной используется запись:

<переменная>::=< символ >...

- Переменные формируются из любого количества символов, кроме знаков (,), точка, двоеточие, пробел, перевод строки. **Первым символом должна быть буква.**
- **Переменная – это объект, имеющий имя и значение.** Имя служит для обозначения объекта. Правила записи переменных, приведенные выше, это, по сути дела, правила записи имен.
- Конкретное значение переменная приобретает в процессе выполнения программы, например, переменная, имеющая имя *XX*, может последовательно принимать значения 10, 12.3, «НАЧАЛО», *NIL* ...
- Помимо перечисленных атомов определен еще «пустой атом», обозначаемый как *NIL*, что означает «ничего», т.е. атом, не имеющий никакого значения.

ПРИМЕРЫ

Правильной записи данных

- 0, -123, 0.0 – целые константы;
- 12.3, -12.34E8 – действительные константы;
- “ABCD”, “Укажите точку”, “\nУкажите точку” – текстовые константы;
- A1, ABCD, CROSS_POINT_2 – переменные (имена переменных);
- NIL – пусто.

Неправильной записи данных

- -100000 – целая константа за пределами допустимых значений;
- .25 – действительная константа без целой части;
- 25. – действительная константа без дробной части;
- “AB\CD” – управляющий символ “\C” не определен, символ “\” должен повторяться дважды;
- “ABCD” – отсутствуют закрывающие кавычки;
- AB CD – такая запись возможна, но это уже не имя переменной, т.к. внутри имеется пробел.

07.10.2024

ПОСТРОЕНИЕ ПРОСТЫХ ГРАФИЧЕСКИХ ПРИМИТИВОВ

Раздел 2

Любая команда графического редактора может быть включена в программу на *Visual LISP*, что позволяет строить изображения из программы автоматически или с участием пользователя.

- Это делается с помощью **функции *COMMAND***, имеющей вид
- ***(COMMAND “< имя команды >” [< аргумент >...])***
- Здесь *<имя команды >* — имя требуемой команды, определенной в данной версии графического редактора.
- **Функция *COMMAND* является очень важной функцией, которая имитирует ввод пользователя в командной строке графической системы с клавиатуры.**

Спасибо за внимание!

